DS 642: Applications of Parallel Computing

Shahriar Afkhami

Week 14:
Final Project and Concluding Thoughts

Project Report

Project Goal: understand performance

- Provide description of your problem.
  - A description of the design choices that you tried and how did they affect the performance.
- Describe performance analysis, including:
  - Serial tuning
  - Strong and weak scaling study
  - Speedup plots
  - Profiling of communication and computation
  - Parallelization strategy (communication, synchronization, etc.)
  - Comparison to analytical models
- Future work/direction
- The organization of the code:
  - Provide comments in the code.
  - Describe error handling and provide tests.

Goals

Understand reasons about code performance and optimization:

- Hardware
  - Tune existing codes for modern hardware
- Software
  - Apply good software practices
- Algorithm
  - Good models that are also easy to program

Many factors to consider

- Pipelining, concurrent execution, and vectorization
- Memory hierarchy and the cost of data movement
- Communication costs (latency and bandwidth)
- Synchronization overheads

Strategies

Numerical ideas and improving performance:

- Ideas of sparsity and locality
  - Cache-friendly, compressed sparse formats
- Domain decomposition
  - Graph partitioning and communication / computation ratios
- Building on top the existing libraries
  - BLAS, LAPACK, ScaLAPACK, $\cdots$
- Numerical solvers and associated programming patterns
  - Sparse versus dense matrices
  - Direct versus iterative solvers

Parallel environments:

- MPI
  - Portable to many implementations
  - High development complexity
- OpenMP
  - Parallelize C, Fortran codes with simple changes
  - Limited scalability
- CUDA, OpenCL
  - Highly data-parallel kernels (e.g. for GPU)
- Hybridization
  - Leverage the strengths of various hardware tiers

Improving performance:

- Working code and test cases first
- Trade off between performance and development efforts (worry about performance at the bottlenecks)
- Compilers and optimization flags
- Use libraries already tuned
- Debuggers and profilers
- Identify major bottlenecks
- Tune the data layout and algorithms for cache locality
- hardware utilization

Libraries:

- Linear Algebra: LAPACK and BLAS
- Performance: ATLAS, MKL, AMD
- FFTs: FFTW, cuFFT
- Domain Decomposition: METIS, ParMETIS, Zoltan, GraphBLAS
- Frameworks: PETSc, Trilinos
- Collections: Netlib (classic numerical software), ACTS (reviews of parallel code)

Development ideas:

- Traditional HPC (PDEs, Engineering, Optimization)
  - Next-Gen PDE Solvers: Development of matrix-free methods to alleviate memory bandwidth bottlenecks.
  - Focus: Extreme scaling, accuracy, and latency.
  - Coupled Simulations: Developing frameworks for multi-physics simulations that balance load between disparate solvers (e.g., fluid dynamics + structural mechanics).
- Graph Applications
  - Hardware-Aware Graph Processing: Developing graph analytics frameworks (like GraphBLAS implementations) tailored for GPU memory hierarchies.
  - Communication-Efficient Partitioning: Algorithms that reduce inter-node communication for graph traversal.
  - Hybrid Graph/PDE Solvers: Creating workflows that use graph analytics to preprocess meshes for PDE solvers.

Development ideas:

- Statistics/ML/AI
  - I/O-Aware AI Frameworks: Integrating high-performance storage systems directly with AI training frameworks (e.g., optimized data loaders for PyTorch on parallel file systems).
  - Surrogate Modeling (AI-Driven Simulation): Training neural networks to replace expensive PDE solvers in engineering simulations, combining high-accuracy simulation data with fast inference.
  - Mixed-Precision Solvers: Developing linear algebra libraries that maximize the use of low-precision Tensor Cores while maintaining scientific accuracy.
- Big Data Techniques in HPC
  - Data locality, stream processing, flexibility.
  - In-Situ Visualization and Analysis: Processing data while it is still in memory during the simulation, rather than writing to disk and analyzing later (combining DB approaches with simulation).